

Name: _____

Date: _____

Physics 0525: Lab 12

Arduino II

Equipment: Arduino, Sparkfun Inventor's Kit, laptop computer with the Arduino IDE (Integrated Development Environment) installed.

Preliminaries:

In the previous lab you were introduced to the Arduino and the concept of physical computing: Building interactive systems by the use of software and hardware that can sense and respond to the outside (physical) world. This time you will learn about debouncing mechanical switches, hardware and software interrupts, parallel vs. serial communication, and how to install external libraries in the Arduino software.

Part 1: Debouncing Mechanical Switches

Build the circuit shown in Figure 1. The push button with pull-down resistor is connected to input pin 2 of the Arduino. The LED is connected to output on pin 13.

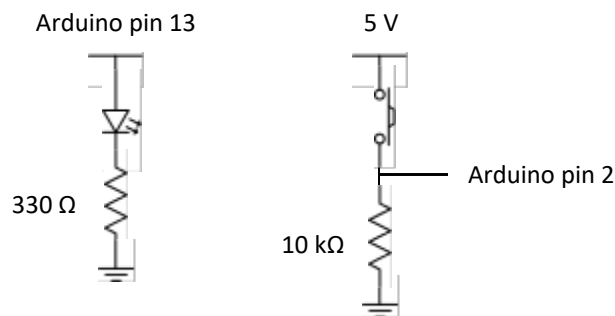


Figure 1. Circuit for button-controlled LED

What is the voltage on the Arduino digital pin 2 when

Button is not pressed: 0.01 [V]

Button is pressed: 4.76 [V]

Remember that the button is “normally open” and the input impedance of the Arduino digital pins is extremely large.

Copy and paste the sketch from Figure 2 into a blank sketch and upload it to the Arduino. Push the button repeatedly. Each button press should toggle the LED on or off. If this is not the case describe what you actually see and give an estimate of the failure rate (e.g. how many times it fails out of 50 trials?).

Name: _____

Date: _____

```
/*
  Debounce Button

  Each time the button pin goes from LOW to HIGH (e.g. because
  of a push-button press), the LED pin is toggled from LOW
  to HIGH or HIGH to LOW. There's a minimum delay between
  toggles to debounce the circuit (i.e. to ignore noise).

  The circuit:
  * LED attached from pin 13 to ground
  * pushbutton attached from pin 2 to +5V
  * 10K resistor attached from pin 2 to ground

  created 21 November 2006
  by David A. Mellis
  modified 30 Aug 2011
  by Limor Fried
  modified 28 Dec 2012
  by Mike Walters
  Modified 05 Feb 2015
  by Everett Ramer

  This example code is in the public domain.

  http://www.arduino.cc/en/Tutorial/Debounce
*/

// Constants won't change:
const int buttonPin = 2;    // the number of the pushbutton pin
const int ledPin = 13;      // the number of the LED pin

// Variables will change:
int buttonState = 0;        // current state of the button
int lastButtonState = 0;    // previous state of the button
int ledState = 0;           // current state of the LED

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // read the pushbutton input pin:
  buttonState = digitalRead(buttonPin);

  // button pushed if buttonState went from LOW to HIGH
  if (buttonState == HIGH && lastButtonState == LOW) {
    // toggle previous LED state
    ledState = !ledState;
    digitalWrite(ledPin, ledState);
    // Delay a little bit to avoid bouncing
    // delay(200);
  }
  lastButtonState = buttonState;
}
```

Figure 2. Debounce button sketch

Name: _____

Date: _____

Mechanical switches are prone to bouncing; that is, when a switch is closed the contacts vibrate making and breaking contacts for a short time which might last several milliseconds after closure. You may actually press the button only once, but the bouncing produces a random number of high/low pulses that leave your LED in an unpredictable state depending on whether you had an odd or even number of bounces. Connect pin 2 to your oscilloscope and observe what happens when the button is pressed (zoom in on the time/horizontal axis to see the bounces and use Normal trigger mode).

✓ 25/50 failure rate, 50%

All switches used as digital input to electronic devices, including the keyboard on your laptop, must be debounced. The most primitive form of debouncing is to add a delay after sensing the change in a switch's state. To enable debouncing in the current sketch, uncomment the statement

```
// delay(200);
```

Now press the button repeatedly. Describe the improvement in the reliability of the toggling action?

The reliability of the button has increased dramatically - the switch always seems to work as expected

Explain how this delay debounces the switch?

The delay function adds a cooldown interval between the check state function, ensuring vibrations in the mechanical trigger prevent overwriting the final state assignment at the end of the main

Name: _____

Date: _____

Part 2: Hardware Interrupts

You might have noticed that the sketch in Figure 2 repeatedly checks the state of the button to determine when it is pressed. This is called polling but this is not very efficient way to deal with an external input; the microcontroller cannot do anything else otherwise it may miss the input. Interrupts allow the microcontroller to respond to events immediately without having to poll continuously to see if anything has changed. This can improve responsiveness, particularly if `loop()` contains any time-consuming task.

To demonstrate the use of interrupts we will use the circuit you constructed in Part 1 and the sketches in Figure 3. The LED should light up as long as the button is pressed. The sketch on the left uses polling and the one on the right uses an interrupt.

```
/*
  Uses polling to control LED from
  button.

  Same circuit as Part 1.

  Sketch created by Dave Auld
  */

const int bPin = 2; // button pin
const int lPin = 13; // LED pin
int bState = 0;      // button state

void setup() {
  pinMode(bPin, INPUT);
  pinMode(lPin, OUTPUT);
}

void loop() {
  // read the button pin:
  bState = digitalRead(bPin);
  digitalWrite(lPin, bState);

  // simulate a complex task
  for (int i = 0; i < 100; i++) {
    // do nothing but waste time
    delay(10);
  }
}
```

```
/*
  Uses interrupt to control LED from
  button.

  Same circuit as Part 1.

  Sketch created by Dave Auld
  */

const int bPin = 2;      // button pin
const int lPin = 13;     // LED pin
// global variables used with
// ISR should be volatile
volatile int bState = 0; // button state

void setup() {
  pinMode(bPin, INPUT);
  pinMode(lPin, OUTPUT);
  // Attach interrupt:
  // 0 (interrupt attached to Pin 2)
  // ISR = bEvent
  // mode = CHANGE
  attachInterrupt(0, bEvent, CHANGE);
}

void loop() {
  // simulate a complex task
  for (int i = 0; i < 100; i++) {
    // do nothing but waste time
    delay(20);
  }
}

// Interrupt Service Routine (ISR)
void bEvent() {
  bState = digitalRead(bPin);
  digitalWrite(lPin, bState);
}
```

Figure 3. Interrupt demo sketches: Left with polling, right with interrupt

Name: _____

Date: _____

Describe differences in the behaviors of the two sketches.

Left sketch: adds a larger delay between polling button state as defined by the for loop.

Right sketch: polls the state immediately when the button is toggled and adds a delay before returning. Toggles in off state thus ensuring the LED is always

The syntax for enabling an interrupt is `attachInterrupt(interrupt, ISR, mode)` on when pressed and always off when not.

- The Arduino UNO has two hardware interrupts: 0 (on digital pin 2) and 1 (on digital pin 3).
- Interrupt Service Routine (ISR) is a user defined function, which is executed when an interrupt is detected. Generally, an ISR should be as short and fast as possible. If your sketch uses multiple ISRs, only one can run at a time, other interrupts will be ignored (turned off) until the current one is finished. Since `delay()` and `millis()` both rely on interrupts, they will not work while an ISR is running. `delayMicroseconds()`, which does not rely on interrupts, will work as expected. Typically global variables are used to pass data between an ISR and the main program. To make sure global variables used in an ISR are updated correctly, declare them as volatile.
- Mode determines how the interrupt is triggered
 - LOW to trigger the interrupt whenever the pin is low,
 - CHANGE to trigger the interrupt whenever the pin changes value
 - RISING to trigger when the pin goes from low to high,
 - FALLING for when the pin goes from high to low.

Name: _____

Date: _____

Part 3: Libraries

Libraries are collections of useful software functions. The Arduino IDE comes preloaded with many libraries, which you can see when you click on the **File > Examples** menu (see Figure 4). Many libraries are devoted to specific types of hardware, like **LiquidCrystal** (for alphanumeric LCD displays), **SD** (for reading and writing SD flash memory cards), **Stepper** (for controlling stepper motors), etc.

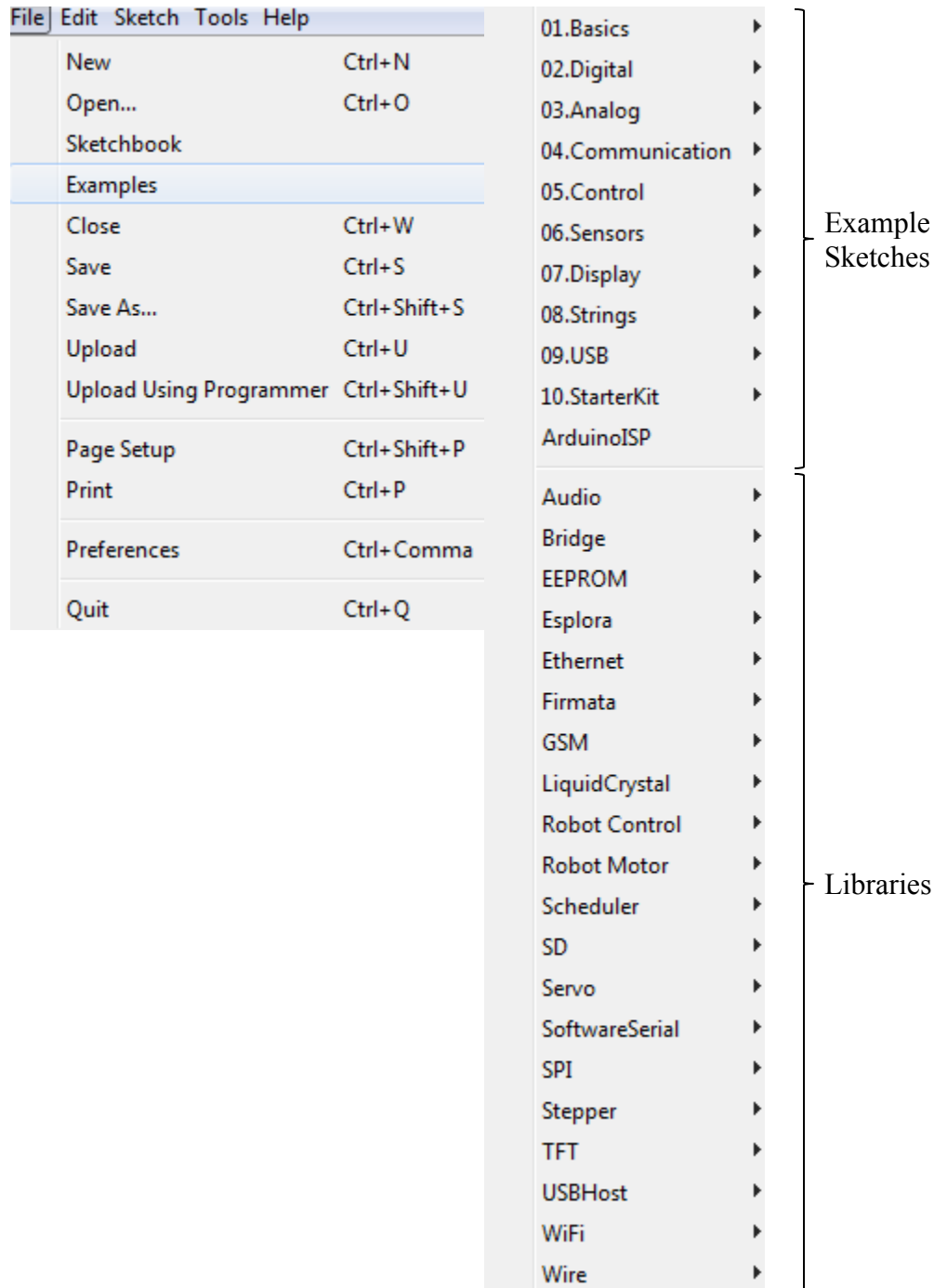


Figure 4. Preloaded libraries in the Arduino IDE.

Name: _____

Date: _____

To use a library in your sketch you need the `#include` directive at the beginning of the sketch. For example, here is part of a sketch that uses the GSM library to send an SMS message from the Arduino GSM shield to a cell phone.

```
/*  
  SMS sender  
  
  This sketch, for the Arduino GSM  
  shield, sends an SMS message...  
  
  created 25 Feb 2012  
  by Tom Igoe  
*/  
  
// Include the GSM library  
#include <GSM.h>  
...
```

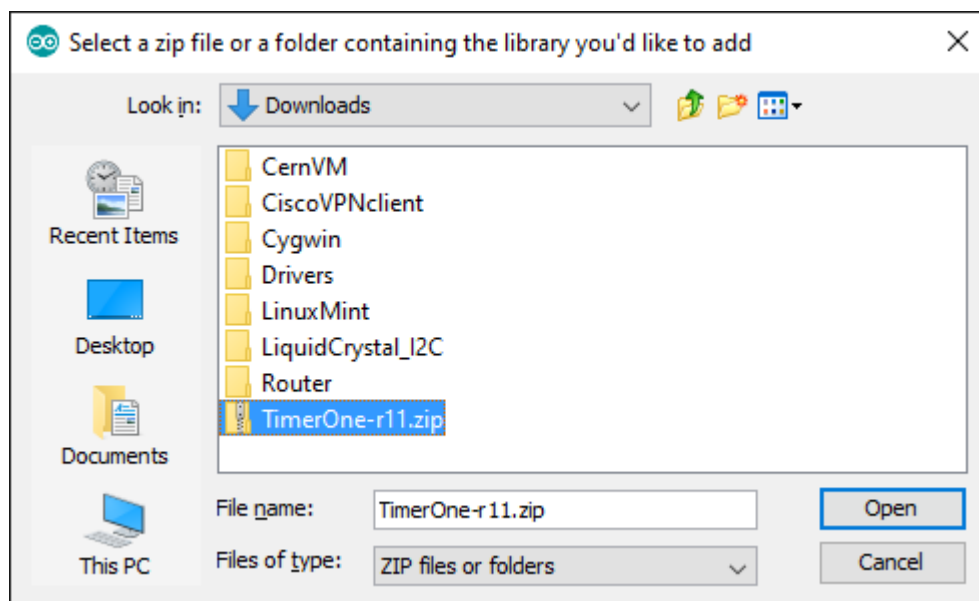
Figure 5. Including an Arduino library in a sketch

New libraries are being created all the time and you will need to install some of them from time to time. In the next part of this experiment, we will need to use the TimerOne library, which is not included in the official Arduino package.

1. Download **TimerOne-r11.zip** file from Canvas to your computer.
2. In the Arduino IDE menu select

Sketch > Include Library > Add .ZIP Library...

3. Locate the TimerOne-r11.zip file on your computer and click open.



Name: _____

Date: _____

4. Return to the File > Examples menu of the Arduino IDE and verify that the TimerOne library appears in a new section for contributed libraries at the bottom of the drop-down list.



Name: _____

Date: _____

Part 4: Timer Interrupts

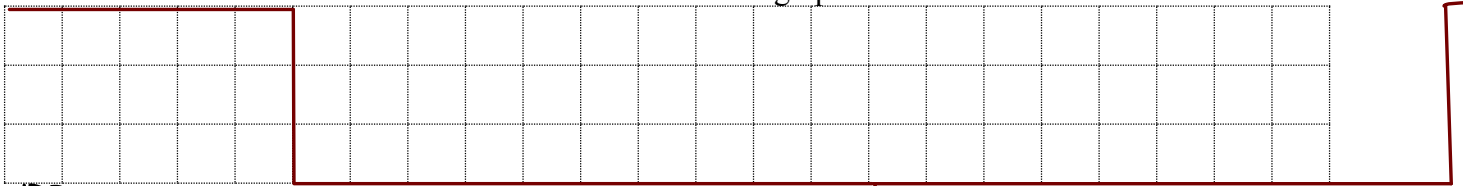
In the previous lab you worked with the Blink sketch that uses `delay()` to control the blink rate of an LED. Open the sketch again (File -> Examples -> 01. Basic -> Blink). Let's suppose you want the LED to blink on and off every 0.5 second by changing the delay in the sketch:

```
void loop() {  
  digitalWrite(13, HIGH); // turn the LED on  
  delay(500);             // wait 0.5 second  
  digitalWrite(13, LOW);  // turn the LED off  
  delay(500);             // wait 0.5 second  
}
```

Suppose you add an additional task in the loop that takes 1.5 seconds to complete.

```
void loop() {  
  digitalWrite(13, HIGH); // turn the LED on  
  delay(500);             // wait 0.5 second  
  digitalWrite(13, LOW);  // turn the LED off  
  delay(500);             // wait 0.5 second  
  
  // additional task that takes 1.5 s  
  delay(1500);  
}
```

Plot the state of the LED as a function of time on the graph below.



100ms

The `delay()` function is described as a blessing and a curse. What problem might you run into if you use it to control the timing of an event?

You have to calculate the time delay manually to ensure timing is correct

Name: _____

Date: _____

Besides hardware interrupts triggered by an external event, one can also trigger an ISR at regular intervals using the TimerOne library you just installed. In this case it does not matter what the sketch is doing between events (turning LED on/off). When the interrupt is triggered the Arduino will suspend whatever it is working on, process the event, and then resume the task it was doing before the interrupt. The events will be precisely timed.

```
#include <TimerOne.h>

const int lPin = 13;    // LED pin
// global variables used with ISR should be volatile
volatile int lState = 0; // button state

void setup() {
  pinMode(lPin, OUTPUT);
  Timer1.initialize(500000); // set 0.5 s between interrupts
  Timer1.attachInterrupt(toggleOutput);
}

void loop() {
  // simulate a complex task
  for (int i = 0; i < 100; i++) {
    // do nothing but waste time
    delay(20);
  }
}

void toggleOutput(){
  digitalWrite(lPin, lState);
  lState = !lState;
}
```

Figure 6. Timer demo sketch

To see this, run the sketch in Figure 6 using the circuit you built for Part 1. Change the delay from 20 ms, to 200 ms, 2000 ms...

Explain why this does not affect blink rate of the LED.

This delay function never is called since the attach interrupt is attached to the timer function.

Part 5: Parallel vs. Serial Communication

Imagine for the moment you have been asked to design a circuit to transmit the character 'A' between two chips in a digital device. The ASCII representation of 'A' is 01000001, so this task will require the transmission of eight bits (binary digits) of information.

Figure 7 illustrates two basic ways of doing this.

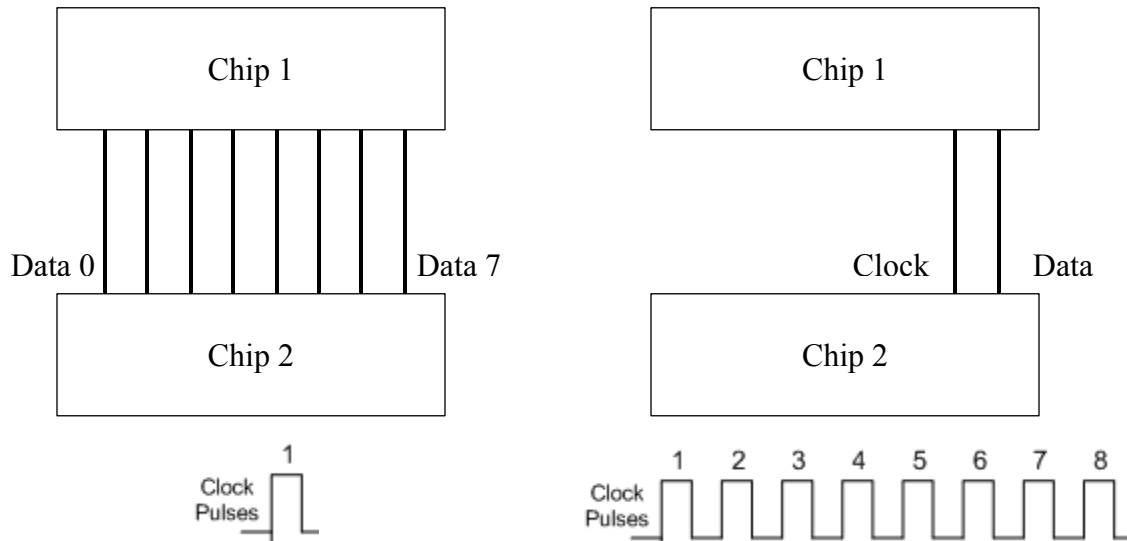


Figure 7. Transmission of data: Parallel (on left) and serial (on right)

In the parallel method shown on the left in Figure 7, all eight bits are transferred simultaneously. In the serial method the bits are transferred one at a time. The serial method is slower but requires fewer wires. Since wires take up space and make circuit construction more difficult, the serial approach is much easier to use than the parallel one.

You may be aware of serial and parallel interfaces that are used to connect computers to printers, external memory devices, a mouse or keyboard, or laboratory instruments. The interfaces we are talking about here are used over very short distances between integrated circuits on the same printed circuit board or within a single device. Two common protocols for such low-level serial communication are I2C (Inter-Integrated Circuit and pronounced "I two C") and SPI (Serial Peripheral Interface).

Parallel Method

The next task is designed to help you appreciate the convenience of serial *versus* parallel. Begin by constructing the 16 x 2 LCD Display circuit described in the centerfold of the **Sparkfun Inventor's Guide** (you can also find it in Canvas). This is the parallel interface circuit. Watch out! There are lots of wires and many opportunities to make mistakes. When this is complete, open the **Blink** sketch found in

File > Examples > LiquidCrystal > Blink

Run **Blink** and describe what happens. If you see a message and the cursor flashes you have wired it correctly. If you do not see the message on the display, try to turn the

Name: _____

Date: _____

potentiometer knob to adjust the contrast of the LCD. Once the display is working, open and upload the **HelloWorld** sketch from the same folder.

On a scale of 1-10, rate the effort required to construct the parallel LCD circuit 1000

Serial Method

Now we will reconfigure the 16 x 2 LCD circuit using the I2C serial protocol. For this we will use a small backpack circuit from DFRobot that translates between serial I2C and parallel. See

[http://www.dfrobot.com/wiki/index.php/I2C/TWI_LCD1602_Module_\(SKU:_DFR0063\)](http://www.dfrobot.com/wiki/index.php/I2C/TWI_LCD1602_Module_(SKU:_DFR0063))

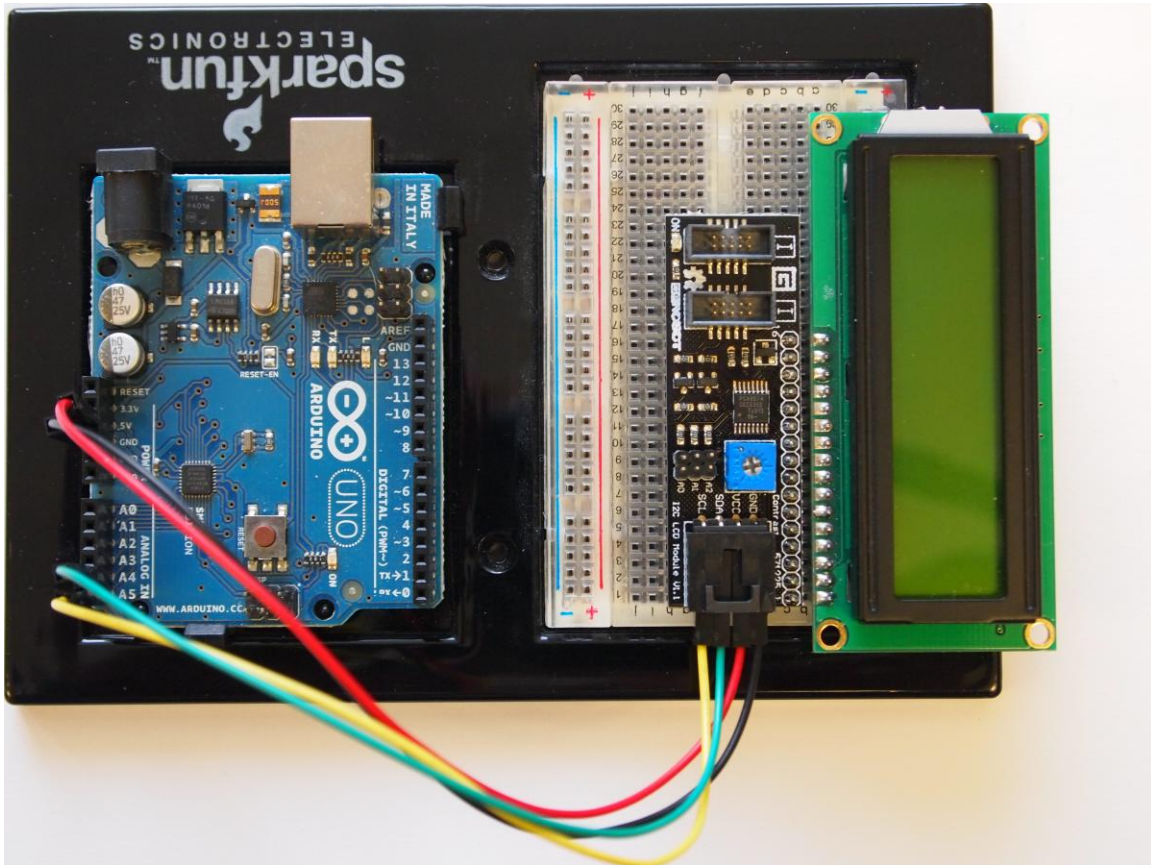


Figure 8. DFRobot I2C-to-parallel backpack

Mount the LCD and I2C backpack on the breadboard so their pins 1-16 are connected (Figure 8). Use four male-to-female jumper wires to connect the following terminals:

Backpack	Arduino UNO
GND	GND
VCC	5 V
DAT (SDA)	I2C Data (pin A4 on the UNO)
CLK (SCL)	I2C Clock (pin A5 on the UNO)

Name: _____

Date: _____

Next install the LiquidCrystal_I2C library in Arduino:

1. Download **LiquidCrystal_I2C.zip** file from Canvas to your computer.

2. In the Arduino IDE menu select

Sketch > Include Library > Add .ZIP Library..

3. Locate the LiquidCrystal_I2C.zip file on your computer and click Open.

4. Go back to File > Examples menu and verify that LiquidCrystal_I2C is added at the bottom of the list.

5. Open the **HelloWorld** sketch found in

File > Examples > LiquidCrystal_I2C > HelloWorld

and upload it to the Arduino.

You can make it display the elapsed time in seconds, like the “HelloWorld” sketch you used a few minutes ago, by adding the following statements to `loop()` in place of the second line of text:

```
// set the cursor to column 0, line 1
lcd.setCursor(0, 1);
// print the number of seconds since reset:
lcd.print(millis() / 1000);
```

On a scale of 1-10, rate the effort required to construct the serial LCD circuit $\lim_{x \rightarrow \infty} |E - x^x|$.

You can buy a 16 x 2 LCD with parallel interface for \$10. The same display with a serial interface will cost you twice that.

